

# Portfolio de SAÉ

Gestionnaire de Notes — Système de gestion de notes étudiants

|              |                                                           |
|--------------|-----------------------------------------------------------|
| Étudiant     | Furkan                                                    |
| Formation    | IUT — Informatique                                        |
| Projet       | Gestionnaire de Notes (SAÉ)                               |
| Équipe       | Furkan, Bahadasht, Akram                                  |
| Période      | 18 mai → 12 juin 2026                                     |
| Technologies | Django 4.2 · Python 3 · MariaDB · Debian 12 · Bootstrap 5 |

## 1. Summary (English)

### Project Overview

The Albert Project is a web-based student grade management system developed as part of a team project (SAÉ) during our IUT training. The application was built using Django 4.2 as the web framework and MariaDB as the database, hosted on a dedicated Debian 12 virtual machine.

The system allows teachers to manage students, teaching units (UE), resources, exams, and grades through a complete CRUD interface. Grades can be entered manually or imported via a CSV file. The application also generates individual grade transcripts with weighted averages.

My personal contributions included designing the database models, implementing the student CRUD module (including photo upload), configuring the Django-MariaDB connection, setting up the authentication system with role-based access control, and performing functional testing across the full application.

## 2. Contexte du projet

Dans le cadre de notre formation IUT, nous avons réalisé une SAÉ (Situation d'Apprentissage et d'Évaluation) consistant à développer une application web complète de gestion de notes étudiants, appelée Gestionnaire de Notes.

Le cahier des charges imposait la gestion de six entités de données : les étudiants, les unités d'enseignement (UE), les ressources associées aux UE, les enseignants, les examens et les notes. Pour chacune de ces entités, un CRUD complet devait être implémenté.

L'application devait également permettre la saisie des notes de deux façons : manuellement via un formulaire tableau, ou par chargement d'un fichier CSV. Enfin, un relevé de notes individuel devait être générable pour chaque étudiant, avec des moyennes pondérées calculées par ressource et par UE.

### Architecture choisie

| Composant       | Technologie         | Rôle                                |
|-----------------|---------------------|-------------------------------------|
| Application web | Django 4.2 (Python) | Framework MVC, vues, templates, ORM |

|                 |                        |                                      |
|-----------------|------------------------|--------------------------------------|
| Base de données | MariaDB 10.11          | Stockage persistant sur VM Debian 12 |
| Front-end       | Bootstrap 5 + HTML/CSS | Interface utilisateur responsive     |
| Déploiement     | Gunicorn + Nginx       | Serveur de production sur VM Linux   |

### 3. Tâches réalisées personnellement

Le travail a été réparti entre les trois membres de l'équipe selon les phases du Gantt. Voici le détail de mes contributions personnelles :

| Phase               | Tâche                                                          | Description                                                                                                                                                 |
|---------------------|----------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Phase 1<br>20/05    | Conception des modèles de données (models.py)                  | Analyse du cahier des charges et conception des 6 modèles Django avec leurs relations (ForeignKey). Exécution des migrations pour créer les tables en base. |
| Phase 2<br>28/05    | CRUD Étudiants (vues + urls + templates)                       | Développement complet des vues, URLs et templates pour la gestion des étudiants. Implémentation de l'upload de photo (ImageField + Pillow + MEDIA_ROOT).    |
| Phase 3<br>01/06    | Configuration Django-MariaDB (settings.py, requirements.txt)   | Paramétrage de la connexion à la VM MariaDB dans settings.py. Mise à jour de requirements.txt et vérification de la migration.                              |
| Phase 4<br>03-04/06 | Authentification & droits élèves (login/logout + accès relevé) | Implémentation du système de connexion/déconnexion. Restriction des élèves à leur seul relevé de notes personnel.                                           |
| Phase 5<br>08-09/06 | Tests fonctionnels & nettoyage (corrections + code propre)     | Tests manuels de l'ensemble des fonctionnalités CRUD. Correction des bugs détectés et nettoyage du code avant rendu.                                        |

### 4. Compétences acquises & éléments de preuve

Chaque tâche réalisée a permis d'acquérir ou de consolider des compétences concrètes. Voici l'analyse réflexive de mes apprentissages :

|                      |                                                   |
|----------------------|---------------------------------------------------|
| <b>Compétence C1</b> | <b>Concevoir un modèle de données relationnel</b> |
|----------------------|---------------------------------------------------|

|                 |                                                                                                                                                                                                                                                                                    |
|-----------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Acquis :</b> | J'ai conçu les 6 entités du projet (Etudiant, UE, Ressource, Enseignant, Examen, Note) en identifiant les attributs de chaque entité et en définissant les relations entre elles via les ForeignKey de Django. Ce travail préalable a structuré tout le développement de l'équipe. |
| <b>Preuve :</b> | Le fichier models.py avec les 6 classes, les ForeignKey et les contraintes (unique_together sur Note). Les 16 tables créées en base après migrate en sont la preuve directe.                                                                                                       |

|                      |                                                                                                                                                                                                                                                               |
|----------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Compétence C2</b> | <b>Développer une application web MVC avec Django</b>                                                                                                                                                                                                         |
| <b>Acquis :</b>      | J'ai maîtrisé le pattern MVC de Django en développant les vues (ListView, CreateView, UpdateView, DeleteView), les formulaires, les URLs nommées et les templates Bootstrap pour l'entité Etudiant. J'ai également géré l'upload de fichiers avec ImageField. |
| <b>Preuve :</b>      | Les fichiers views.py, urls.py et templates/etudiants/ contenant les 4 opérations CRUD complètes. La fonctionnalité d'upload de photo en est un exemple concret.                                                                                              |

|                      |                                                                                                                                                                                                                                   |
|----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Compétence C3</b> | <b>Configurer une architecture client-serveur séparée</b>                                                                                                                                                                         |
| <b>Acquis :</b>      | J'ai configuré la connexion Django vers une base MariaDB distante hébergée sur une VM Debian 12 séparée. Cela impliquait de comprendre les paramètres réseau (HOST, PORT), le driver mysqlclient, et la gestion des requirements. |
| <b>Preuve :</b>      | Le fichier settings.py avec le bloc DATABASES configuré sur l'IP 10.128.206.187:3306 et le requirements.txt. La réussite du migrate vers la VM en est la validation.                                                              |

|                      |                                                                                                                                                                                                                            |
|----------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Compétence C4</b> | <b>Implémenter un système d'authentification et de contrôle d'accès</b>                                                                                                                                                    |
| <b>Acquis :</b>      | J'ai développé les vues de connexion/déconnexion avec le système d'authentification intégré de Django, et mis en place une restriction d'accès par rôle : les élèves ne peuvent consulter que leur propre relevé de notes. |
| <b>Preuve :</b>      | Les décorateurs @login_required sur les vues protégées, la vérification request.user dans la vue relevé, et la page login.html avec deux profils distincts.                                                                |

|                      |                                                                                                                                                                                                                                        |
|----------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Compétence C5</b> | <b>Tester et valider une application web</b>                                                                                                                                                                                           |
| <b>Acquis :</b>      | J'ai réalisé une phase de tests fonctionnels manuels sur l'ensemble des fonctionnalités CRUD, vérifié les droits d'accès selon les profils, et corrigé les bugs identifiés. J'ai également nettoyé le code pour le rendre maintenable. |
| <b>Preuve :</b>      | Les corrections apportées dans le code source, la suppression des fichiers inutiles et la cohérence finale de l'application lors de la démonstration.                                                                                  |

## 5. Analyse réflexive

### Ce que j'ai appris

Ce projet m'a permis de comprendre concrètement le fonctionnement d'une application web complète, de la conception du modèle de données jusqu'au déploiement. La séparation entre l'application et la base de données sur deux machines distinctes m'a donné une vision réelle de l'architecture des systèmes d'information en entreprise.

### **Les choix effectués**

J'ai choisi de commencer par la conception des modèles car c'est la fondation de tout le projet. Un modèle mal conçu aurait impacté tous les développements suivants. Pour le CRUD Étudiants, j'ai utilisé les class-based views de Django plutôt que des vues fonctionnelles, ce qui permet un code plus concis et réutilisable.

### **Les difficultés surmontées**

La principale difficulté a été le débogage du code : certaines erreurs étaient difficiles à identifier et à corriger, notamment dans les vues Django et les templates. Face à ces problèmes, j'ai appris à lire attentivement les messages d'erreur, à tester le code étape par étape et à chercher des solutions dans la documentation officielle de Django.

### **Progression des compétences**

Avant ce projet, j'avais des bases en Django mais jamais développé une application complète en équipe avec une base de données distante. Ce projet m'a permis de progresser significativement sur la gestion de projet en équipe, la configuration d'une architecture réelle, et la rigueur nécessaire pour livrer un code testé et propre.